

فاز ۱۵: DevOps، Docker، Kubernetes و

ابزارهای مرتبط

۱۵.۱ DevOps چیست؟

تعریف ساده: ترکیب "Development" و "Operations". به فرهنگ و مجموعه‌ای از روش‌ها برای:

• خودکارسازی فرآیندهای نرم‌افزاری

• کاهش فاصله بین تیم توسعه و تیم عملیات

• تحویل سریع‌تر و مطمئن‌تر نرم‌افزار

چرخه DevOps:

1 Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (بازگشت به Plan)

۱۵.۲ Docker (ظروف مجازی)

چیست؟ به پلتفرم برای بسته‌بندی برنامه‌ها و وابستگی‌هاشون در به Container (ظرف) که می‌تونه روی هر سیستمی اجرا بشه.

تشبیه: کانتینر مثل به کانتینر حمل و نقل (کشتی) هست که برنامه و همه چیز مورد نیازش داخلش هست.

Virtual Machine با Docker فرق

ویژگی	Docker (Container)	Virtual Machine
سیستمعامل	مشارک با Host	مجزا (هر VM به OS داره)
اندازه	سبک (MB)	سنگین (GB)
سرعت اجرا	سریع	کند
مصرف منابع	کم	زیاد

Dockerfile (دستور ساخت کانتینر)

```

1 # a. انتخاب /Base Image
2 FROM openjdk:17-jdk-alpine
3
4 # b. تنظیم متغیرهای محیطی
5 ENV APP_HOME=/app
6
7 # c. ایجاد دایرکتوری
8 WORKDIR $APP_HOME
9
10 # d. کپی کردن فایل ها
11 COPY target/myapp.jar app.jar
12
13 # e. پورت برنامه
14 EXPOSE 8080
15

```

16 # f. دستور اجرا

17 ENTRYPOINT ["java", "-jar", "app.jar"]

دستورات مهم Docker

1 # a. ساخت Image

2 docker build -t myapp:latest .

3

4 # b. اجرای Container

5 docker run -p 8080:8080 myapp:latest

6

7 # c. های در حال اجرا Container مشاهده

8 docker ps

9

10 # d. ها Container مشاهده همه

11 docker ps -a

12

13 # e. Container توقف

14 docker stop container_id

15

16 # f. Container حذف

17 docker rm container_id

18

19 # g. ها Image مشاهده

20 docker images

21

22 # h. ورود به Container

23 docker exec -it container_id /bin/sh

24

25 # i. مشاهده لاگ ها

26 docker logs container_id

Docker Compose (اجرای چند کانتینر با هم)

1 # docker-compose.yml

2 version: '3.8'

3

4 services:

5 # اپلیکیشن / Spring Boot

6 app:

7 build: .

8 ports:

9 - "8080:8080"

10 depends_on:

11 - db

12 environment:

13 - SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/mydb

14

15 # دیتابیس

16 db:

17 image: mysql:8.0

18 environment:

```
19 - MYSQL_ROOT_PASSWORD=root
20 - MYSQL_DATABASE=mydb
21 ports:
22 - "3306:3306"
23
24 # Redis برای کش
25 redis:
26 image: redis:alpine
27 ports:
28 - "6379:6379"
29 # اجرای همه سرویس‌ها
30 docker-compose up -d
31
32 # توقف همه سرویس‌ها
33 docker-compose down
```

Kubernetes (K8s) ۱۵.۳

چیست؟ سیستم مدیریت Container ها در مقیاس بزرگ. مثل یک Captin برای کشتی‌های (Containerهای) Docker.

مفاهیم کلیدی:

مفهوم	توضیح	تشبیه
Pod	کوچک‌ترین واحد، شامل یک یا چند Container	یه سرباز
Deployment	مدیریت Pod ها (تعداد، بروزرسانی)	یه گروه سرباز
Service	دسترسی به Pod ها از بیرون	آدرس پادگان
Ingress	مدیریت ورودی (مسیریابی)	ورودی اصلی
ConfigMap	ذخیره تنظیمات	تابلوی اعلانات
Secret	ذخیره اطلاعات حساس (پسورد)	صندوق امانات

مثال ساده Kubernetes

```

1 # deployment.yaml
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: myapp-deployment
6 spec:
7   replicas: 3 # اجرا کن Pod تا ۳
8   selector:
9     matchLabels:
10      app: myapp
11   template:
12     metadata:
13       labels:
14         app: myapp
15   spec:
16     containers:
17       - name: myapp

```

```
18   image: myapp:latest
19   ports:
20     - containerPort: 8080
21 ---
22 # service.yaml (برای دسترسی از بیرون)
23 apiVersion: v1
24 kind: Service
25 metadata:
26   name: myapp-service
27 spec:
28   selector:
29     app: myapp
30   ports:
31     - port: 80
32       targetPort: 8080
33   type: LoadBalancer
34 # a. اعمال به کلاستر
35 kubectl apply -f deployment.yaml
36 kubectl apply -f service.yaml
37
38 # b. Pod ها مشاهده
39 kubectl get pods
40
41 # c. Service ها مشاهده
42 kubectl get services
43
```

44 # d. **Deployment** مشاهده

45 `kubectl get deployments`

46

47 # e. **Pod** تغییر تعداد (مقیاس دهی)

48 `kubectl scale deployment myapp-deployment --replicas=5`

Apache Kafka ۱۵.۴

چیست؟ به سیستم پیام‌رسانی توزیع‌شده (Distributed Messaging System) برای پردازش جریان داده (Stream Processing).

تشبیه: مثل به سیستم حمل و نقل عمومی (مترو) که پیام‌ها رو بین Producerها (فرستنده) و Consumerها (گیرنده) جابه‌جا می‌کنه.

مفاهیم کلیدی:

مفهوم	توضیح
Producer	فرستنده پیام
Consumer	گیرنده پیام
Topic	دسته‌بندی پیام‌ها (مثل کانال)
Partition	تقسیم Topic به چند بخش برای مقیاس‌پذیری
Broker	سرور Kafka

چه موقع از Kafka استفاده کنیم؟

• **Event Sourcing**: ثبت همه رویدادها

• **Log Aggregation**: جمع‌آوری لاگ‌ها از چندین سرویس

•Stream Processing: پردازش لحظه‌ای داده (مثل تحلیل لحظه‌ای)

•Message Queue: ارتباط بین میکروسرویس‌ها

مثال ساده در جاوا (با Spring Boot)

```

1 // Producer - فرستنده
2 @Service
3 class KafkaProducerService {
4     @Autowired
5     private KafkaTemplate<String, String> kafkaTemplate;
6
7     public void sendMessage(String message) {
8         kafkaTemplate.send("my-topic", message);
9     }
10 }
11
12 // Consumer - گیرنده
13 @Service
14 class KafkaConsumerService {
15     @KafkaListener(topics = "my-topic", groupId = "my-group")
16     public void listen(String message) {
17         System.out.println("Received: " + message);
18     }
19 }

```

۱۵.۵ Linux (لینوکس) – دستورات مهم برای بک‌اندکار

دستورات حیاتی که باید بلد باشی:

۱. مدیریت فایل‌ها و دایرکتوری‌ها

```
1 ls -la      # نمایش همه فایل‌ها با جزئیات
2 cd /path    # تغییر دایرکتوری
3 mkdir newdir # ساخت دایرکتوری جدید
4 rm -rf dir   # حذف دایرکتوری (با احتیاط!)
5 cp file1 file2 # کپی فایل
6 mv file1 file2 # جابجایی یا تغییر نام
```

۲. مشاهده محتوای فایل

```
1 cat file.txt # نمایش کامل فایل
2 less file.txt # نمایش صفحه‌به‌صفحه
3 tail -f log.txt # (Live) دنبال کردن لاگ
4 head -n 10 file # خط اول ۱۰
```

۳. مدیریت فرآیندها (Process)

```
1 ps aux      # مشاهده همه فرآیندها
2 top         # مشاهده مصرف منابع (لایو)
3 kill -9 PID # کشتن یک فرآیند
4 nohup java -jar app.jar & # اجرا در پس‌زمینه
```

۴. مدیریت شبکه

```

1 ping google.com # تست اتصال
2 netstat -tulpn # مشاهده پورت‌های باز
3 curl http://api.test # HTTP درخواست
4 ss -tuln # مشاهده سوکت‌ها

```

۵. مدیریت دیسک و حافظه

```

1 df -h # فضای دیسک
2 du -sh * # حجم پوشه‌ها
3 free -h # RAM حافظه

```

! سوال مصاحبه‌ای: فرق apt-get و yum چیه؟

Feature	apt-get	yum
توزیع	Debian/Ubuntu	RedHat/CentOS
مخزن	APT	RPM
دستور نصب	apt-get install	yum install

۱۵.۶ CI/CD (Continuous Integration / Continuous Deployment)

چیست؟ خودکارسازی فرآیند Build، Test و Deploy کد.

مراحل:

1.Commit: توسعه‌دهنده کد رو Push می‌کنه

2.Build: سرور خودکار کد رو Build می‌کنه

3.Test: تست‌ها اجرا میشن

4.Deploy: برنامه روی سرور Deploy میشه

ابزارهای معروف:

• Jenkins (محبوب‌ترین)

• GitHub Actions

• GitLab CI

• CircleCI

GitHub Actions مثال

```
1 name: Java CI/CD
2
3 on:
4   push:
5     branches: [ main ]
6
7 jobs:
8   build:
9     runs-on: ubuntu-latest
10
11     steps:
12     - uses: actions/checkout@v2
13
```

```

14 - name: Set up JDK 17
15   uses: actions/setup-java@v2
16   with:
17     java-version: '17'
18
19 - name: Build with Maven
20   run: mvn clean package
21
22 - name: Build Docker Image
23   run: docker build -t myapp .
24
25 - name: Deploy to Server
26   run: docker push myapp

```

۱۵.۷ خلاصه یک صفحه ای

ابزار	کاربرد	سطح یادگیری
Docker	بسته بندی برنامه	ضروری ★★★★★
Docker Compose	اجرای چند کانتینر	ضروری ☆★★★★
Kubernetes	مدیریت کانتینرها در مقیاس بزرگ	متوسط ☆☆☆★★
Kafka	پیام رسانی بین سرویس ها	متوسط ☆☆☆★★
Linux	سیستم عامل سرور	ضروری ★★★★★
Jenkins/GitHub Actions	CI/CD	مهم ☆★★★★